Część 1. Podstawa Teoretyczna

Ten przewodnik pomoże Ci zrozumieć, czym są biblioteki **Apache POI** oraz **Apache Commons CSV**, do czego służą w ekosystemie Javy i jakie mają typowe zastosowania. Są to kluczowe narzędzia podczas mapowania danych wejściowych z plików na obiekty Javy (DTO / rekordy) oraz przy generowaniu eksportów.

1. Excel vs CSV – Co wybrać i kiedy?

Zarówno CSV, jak i Excel (.xlsx) służą do przechowywania danych tabelarycznych, jednak ich natura narzuca zupełnie inne scenariusze użycia.

Cechy	CSV (Comma-Separated Values)	Excel (.xlsx)
Struktura i Typy	Płaski tekst, brak typów (wszystko to String), brak zakładek/arkuszy.	Plik binarny (archwium XML), obsługuje arkusze, style, formuły i natywne typy danych.
Wydajność	Lekki, idealny do strumieniowania i wielkich plików (miliony wierszy).	Ciężki dla pamięci, wczytanie całego pliku do RAM może spowodować `OutOfMemoryError`.
Reguła kciuka	**Integracje systemów i ETL** (Extract, Transform, Load). Transfer danych do baz.	**Raporty dla użytkowników biznesowych**. Wymagane formatowanie i czytelność.

2. Zależności i Mapowanie na DTO

Obie biblioteki nie są wbudowane w standardowe JDK, należy je dodać poprzez Maven (grupy `org.apache.poi` oraz `org.apache.commons`). Celem korzystania z nich najczęściej nie jest manipulacja komórkami sama w sobie, ale transformacja danych ze świata zewnętrznego na obiekty Javy. We współczesnej Javie do tego celu idealnie nadają się **Rekordy (Records)** lub tradycyjne **DTO (Data Transfer Objects)**.

Przykładowy rekord, na który będziemy mapować wiersze z obu formatów:

```
public record Person(String name, LocalDate birthDate, BigDecimal salary) {}
```

Użycie `BigDecimal` zamiast `double` dla wartości pieniężnych to kluczowa dobra praktyka – chroni przed utratą precyzji w obliczeniach zmiennoprzecinkowych.

3. Apache Commons CSV

CSV to z pozoru prosty format, co często kusi programistów do ręcznego parsowania wierszy (np. `String.split(";")`). Jest to **poważny błąd**. Ręczne rozdzielanie sypie się natychmiast, gdy wewnątrz samego pola tekstowego pojawi się znak separatora, cudzysłów lub znak nowej linii. Od tego są biblioteki takie jak Commons CSV.

Odczyt plików i Dialekty

Biblioteka opiera się na definicji **Dialektu (CSVFormat)**. Domyślnie formatuje się go przez Buildera, gdzie możemy wskazać separator, nagłówki czy zasady cytowania.

```
CSVFormat format = CSVFormat.DEFAULT.builder()
    .setDelimiter(';') // W polskim Excelu separatorem jest często średnik!
    .setHeader()       // Deklaracja, że plik posiada nagłówki
    .withFirstRecordAsHeader() // Użycie pierwszej linii jako mapy nagłówków
    .withIgnoreEmptyLines(true)
    .build();

try (Reader reader = new FileReader("data.csv", StandardCharsets.UTF_8);
     CSVParser parser = format.parse(reader)) {

    // Walidacja obecności kolumny (ochrona przed wybuchem kodu)
    if (!parser.getHeaderMap().containsKey("salary")) {
        throw new IllegalArgumentException("Brak wymaganej kolumny 'salary'");
    }

    for (CSVRecord record : parser) {
        String name = record.get("name");
        // ... parsowanie liczb (z uwzględnieniem locale/kropek) i dat
    }
}
```

Typowe pułapki w CSV

- **Zły delimiter:** Polski Excel często zapisuje CSV ze średnikiem (`;`), a nie przecinkiem. Standard `.RFC4180` oczekuje przecinka. Trzeba o to dbać przy użyciu `setDelimiter()`.
- **Kodowanie i znacznik BOM:** Jeśli używasz `UTF-8` (co jest dobrą praktyką), pamiętaj, że Windowsowy Excel często wymaga na początku pliku znacznika BOM (Byte Order Mark), aby poprawnie wyświetlać polskie znaki diakrytyczne.
- **Różne nazwy nagłówków:** Różnica wielkości liter (`Salary` vs `salary`) powoduje błędy przy `record.get()`. Warto użyć mapowania aliasów.

> **Mini-test wejścia (Sanity check):** Przy odczycie wielkich plików, dobrą praktyką jest wczytanie pierwszych 10 rekordów, zalogowanie licznika wierszy i zweryfikowanie unikalnych wartości kluczowych kolumn. Unikniesz przetwarzania 2 GB zepsutych danych.

4. Apache POI – Zrozumieć strukturę Excela

Apache POI operuje na obiektowym modelu dokumentu. Dla plików `.xlsx` używa się implementacji rozpoczynających się od XSSF. Hierarchia klas jest logiczna i odzwierciedla strukturę arkusza kalkulacyjnego:

`Workbook` (Cały plik) → `Sheet` (Pojedyncza zakładka) → `Row` (Wiersz) → `Cell` (Komórka).

Odczyt danych i Problem z Typami

Użytkownicy w Excelu mogą wpisać dosłownie wszystko. Aby zminimalizować ryzyko `IllegalStateException` podczas prób pobrania liczby z pola, w którym ktoś wstawił spację, używamy **DataFormattera**.

```
Workbook workbook = new XSSFWorkbook(new FileInputStream("data.xlsx"));
Sheet sheet = workbook.getSheetAt(0);
DataFormatter formatter = new DataFormatter(); // "Cywilny" odczyt

for (Row row : sheet) {
    if (row.getRowNum() == 0) continue; // Pomijamy nagłówek

    Cell nameCell = row.getCell(0);
    // Odczyt komórki dokładnie tak, jak widzi ją człowiek w Excelu:
    String nameText = formatter.formatCellValue(nameCell);

    Cell dateCell = row.getCell(1);
    // Jeśli data to "prawdziwa" data w Excelu, pobieramy ją poprawnie:
    LocalDateTime date = dateCell.getLocalDateTimeCellValue();
}
```

Daty, Formuły i Kasa

Excel pod spodem przechowuje **daty jako liczby zmiennoprzecinkowe** (ilość dni od 1900 roku). Jeśli wywołasz `getNumericCellValue()` na komórce z datą, dostaniesz dziwną liczbę. Dlatego zawsze korzystaj z `getLocalDateTimeCellValue()` (dostępne w nowszych wersjach POI, na co wskazuje dokumentacja).

Formuły (np. `=A1+B1`) nie trzymają wyniku jako prostej wartości obok równania w obiekcie komórki. Do ich bezpiecznego odczytania należy wywołać `FormulaEvaluator`, w przeciwnym razie odczytasz sam wzór tekstowy (lub zbuforowaną, nieświeżą wartość).

5. Zapis w POI i praca z gigantycznymi plikami

Jeśli spróbujesz zapisać 2 miliony rekordów do Excela używając standardowego `XSSFWorkbook`, aplikacja pochłonie cały RAM i zakończy się awarią. Apache POI oferuje tryb **Streaming (SXSSF)** przeznaczony specjalnie do obsługi gigantycznych plików (powyżej 100 tysięcy wierszy).

```
// "100" to rozmiar okna - w pamięci trzyma tylko 100 wierszy naraz
try (SXSSFWorkbook sxssfWorkbook = new SXSSFWorkbook(100)) {
    Sheet sheet = sxssfWorkbook.createSheet("Raport");
    // ... generowanie milionów wierszy

    try (FileOutputStream out = new FileOutputStream("wielki_raport.xlsx")) {
        sxssfWorkbook.write(out);
    }
    // Ważne! Czyszczenie plików tymczasowych z dysku:
    sxssfWorkbook.dispose();
}
```

Zapis wartości pieniężnych do komórek to osobny problem. Nie korzystaj z wbudowanego `double`, bo obetniesz grosze. Bezpieczniej zapisać pieniądze konwertując `BigDecimal` na sformatowany String lub nadając komórce natywny styl księgowy z wymuszonym formatowaniem (`Style + DataFormat`).

6. Podsumowanie Dobrych Praktyk

- **CSV - Zawsze jawne parametry:** W kodzie w jednym miejscu trzymaj konfigurację: ustalony delimiter (najlepiej jako stała) i obsługę BOM dla plików wychodzących do Excela Windows.
- **CSV - Streamuj wejście:** Zawsze przetwarzaj odczyt iteracyjnie. Nie ładuj zawartości parsera do `List<Record>`, bo przy plikach ETL rzędu setek megabajtów skończy Ci się pamięć.
- **POI - Optymalizuj style:** Liczba formatowań w jednym pliku .xlsx jest limitowana. Twórz obiekty `CellStyle` (np. pogrubienie, kolory) na poziomie skoroszytu (Workbook), a nie dla każdej pojedynczej komórki!
- **POI - Zamknięcie zasobów:** Zarówno `Workbook`, jak i strumienie plikowe, muszą być otwierane w blokach `try-with-resources`. Zamknięcie skoroszytu natychmiast zwalnia bufory.

Część 2. Propozycja Prezentacji (Referatu)

Poniżej znajduje się scenariusz na 10–12 slajdów, zoptymalizowany dla 2-3 osób. Treści na slajdach są maksymalnie zwarte (w punktach), a rozszerzenia znajdują się w "Notatkach".

Slajd 1: Apache POI i Commons CSV – wstęp i cel Osoba 1

- Cel: Wygodne mapowanie plików `.xlsx` oraz `.csv` na obiekty Javy (DTO / record).
- Problem 1: Parsowanie tekstu kryje mnóstwo pułapek.
- Problem 2: Pliki Excela to w rzeczywistości skompresowane paczki XML.
- Rozwiązanie: Stabilne biblioteki Open Source.

Notatka: Przywitaj grupę. Wyjaśnij, że dzisiaj skupimy się na tym, jak odczytywać i zapisywać dane do/z plików, tak aby uniknąć częstych błędów w produkcyjnym kodzie. Wspomnij o wymaganiach Mavena do dodania obu bibliotek.

Slajd 2: Kiedy CSV, a kiedy Excel? Osoba 1

- **CSV (Comma-Separated Values):**
  - Lekki, szybki w przetwarzaniu, łatwy do strumieniowania.
  - Idealny do: Integracji systemów, procesów ETL (Extract-Transform-Load).
- **Excel (.xlsx):**
  - Obsługuje typy natywne, arkusze, style wizualne i formuły.
  - Idealny do: Raportów końcowych dla użytkowników biznesowych.

Notatka: Omów "Złotą regułę" – jeżeli proces rozmawia z innym systemem, użyj CSV. Jeżeli plik na końcu procesu otwiera księgowa, wygeneruj ładnego Excela z odpowiednio sformatowanymi komórkami.

Slajd 3: Apache Commons CSV – odczyt danych Osoba 1

- Definiowanie formatu za pomocą dialektów (`CSVFormat.DEFAULT` , `.RFC4180`).
- Kluczowe modyfikatory Buildera:
  - `.setDelimiter(';')`
  - `.setHeader()`
  - `.withFirstRecordAsHeader()`
- Iteracyjne czytanie: parser zwraca `CSVRecord` , z którego mapujemy pola po nazwie kolumny.

Notatka: Pokazać fragment kodu (np. w IDE lub rzucając na slajd zaledwie 3-4 linie). Zaakcentować, że iteracyjne czytanie to mus – nie ładujemy gigabajtowego CSV do pojedynczej Listy przed przetwarzaniem.

Slajd 4: Pułapki formatu CSV Osoba 1

- **Ręczne parsowanie:** `String.split(",")` to gwarancja wybuchu, gdy ktoś wpisze przecinek lub enter wewnątrz pola.
- **Zły separator:** Polski Excel domyślnie używa średnika (`;`) dla formatu CSV!
- **Brak BOM (Byte Order Mark):** Excel na Windowsie często potraktuje plik UTF-8 jako ASCII, jeśli nie dodamy na początku pliku nagłówka BOM (krzaczki zamiast znaków pl).
- **Brakujące nagłówki:** Należy używać `headerMap.containsKey("...")` dla sanity checku.

Notatka: Opowiedz anegdotę o tym, że biblioteka Commons CSV doskonale radzi sobie z cytowaniem znaków nowej linii wewnątrz komórek, co czyni pisanie własnego kodu rozdzielającego całkowicie pozbawionym sensu.

Slajd 5: Apache POI – Architektura (Odczyt) Osoba 2

- Podejście zorientowane obiektowo. Logiczna struktura:
- `Workbook` (cały plik) → `Sheet` (arkusz) → `Row` (wiersz) → `Cell` (pojedyncza komórka).
- **DataFormatter:** klasa ratująca życie. Wyciąga zawartość jako `String`, zachowując wizualne formatowanie nałożone przez Excela.

Notatka: Tu zaczyna osoba 2. Pokaż, w jaki sposób POI modeluje plik, wspominając, że iterowanie po komórkach wymaga ostrożności, bo np. wiersz może być fizycznie 'null', jeżeli nic w nim nie ma.

Slajd 6: Excelowe Daty i Formuły Osoba 2

- **Daty:** Excel widzi datę jako ilość dni (liczba zmiennoprzecinkowa). Nie rzutuj w ciemno! Użyj dedykowanej metody: `cell.getLocalDateTimeCellValue()`.
- **Formuły (np. =SUMA(A1:A5)):**
  - Wartość nie aktualizuje się automatycznie.
  - Do poprawnego odczytu wartości wyliczonej potrzebny jest: `FormulaEvaluator`.

Notatka: Mocno zaakcentuj problem dat. Niedoświadczony deweloper traci tu godziny na zrozumienie, dlaczego parser zwraca dziwną liczbę `44200.5` zamiast jego wpisanej w arkuszu daty urodzin.

Slajd 7: Apache POI – Zapis dużych plików (SXSSF) Osoba 3 (lub 1)

- Zwykły zapis (`XSSFWorkbook`) trzyma całą strukturę XML w pamięci RAM.
- Raport na milion wierszy wyrzuci `OutOfMemoryError` .
- Rozwiązanie: klasa **SXSSFWorkbook**.
- Tryb strumieniowy (streaming): Zrzuca wygenerowane wiersze na dysk w locie (tzw. zjawisko flushowania okna, np. co 100 wierszy).

Notatka: Tu zaczyna trzeci prowadzący (jeśli macie 3 osoby). Zaznacz, że klasa zaczyna się na "S" (od Streaming). Po zakończeniu pracy trzeba bezwzględnie wywołać `.dispose()`, by pozbyć się ukrytych plików tymczasowych.

Slajd 8: Operacje finansowe i precyzja Osoba 3 (lub 1)

- W Javie wartości pieniężnych **nigdy** nie reprezentuje się typem `double` (błędy zaokrągleń ułamków binarnych). Używamy klas `BigDecimal` .
- Zapis BigDecimal do Excela:
  - Najbezpieczniej: wygenerować do `String` lub...
  - Wstrzyknąć wartość `double` , ale wymusić specjalny `CellStyle` o określonym profilu rachunkowym, aby Excel nie ukrył ułamków.

Notatka: Wytłumacz krótko, na czym polega gubienie precyzji w double i dlaczego w klasach DTO przy danych o pensjach powinni stosować BigDecimal. Systemy księgowe muszą być precyzyjne co do grosza.

Slajd 9: Pułapki stylów w Excelu Osoba 3 (lub 2)

- Generowanie raportu z nałożonymi stylami z poziomu Javy bywa trudne.
- Wielki błąd: Tworzenie nowego `CellStyle` dla każdej komórki w pętli.
- Skutek: Excel ma limit maksymalnej liczby dozwolonych stylów (najczęściej limit = 64 000 instancji).
- Rozwiązanie: Utwórz jeden obiekt `CellStyle` (np. *headerStyle*) globalnie dla dokumentu i jedynie przypisuj referencję komórkom w pętli.

Notatka: Dość specyficzny, rzadki problem, ale świetny do wspomnienia - pokazuje, że grupa rozumie głębokie detale tego API i wymagania dla laboratorium.

Slajd 10: Podsumowanie (Dobre Praktyki) Wszyscy

- Zawsze korzystaj z bloków `try-with-resources` przy czytaniu plików. Zamykanie strumieni to obowiązek.
- Używaj `DataFormatter` do bezpiecznego wczytywania stringów.
- Wczytując wielki plik podetluj sanity-check pierwszych kilku wierszy, aby uniknąć marnowania czasu CPU i walidować wejście.
- Dla plików tekstowych scentralizuj w aplikacji konfigurację formatu (jawny separator, ustawienia cytowania znaków, zaufanie do biblioteki CSV).

Notatka: Krótkie zwięzłe przypomnienie najważniejszych dobrych rad z obu bibliotek (zamknięcie dla wykładu). Zostawić w pamięci kluczowy wniosek: Narzędzia są potężne, ale i groźne, jeżeli zapomina się o pamięci i limitach.

Slajd 11: Pytania i Odpowiedzi Wszyscy

- Dziękujemy za uwagę!
- Materiały i kod źródłowy: `[link_do_repozytorium]`
- Czy macie jakieś pytania?

Notatka: Strona końcowa, opcjonalnie przekaż grupie jeden mały "gotowiec" kodu na start laboratorium.